MIT ACADEMY OF ENGINEERING

INTERNAL ASSESSMENT · IA 3

Jenkins CI/CD + Terraform Mini Project

Automated build, test, local infrastructure provisioning, and deployment workflow

STUDENT	Sahas Nagar
PROGRAMME	B.Tech Computer Engineering
INSTITUTE	MIT Academy of Engineering (MITAOE), Alandi, Pune, Maharashtra
ACADEMIC YEAR	2026–2027
SEMESTER	As applicable to the course
SUBJECT	Cloud Native DevOps
COURSE CODE	2310423T26-27
ASSIGNMENT TYPE	Internal Assessment Activities

Technical implementation report · Prepared for academic assessment

Table of contents

1. Introduction
2. Objectives
3. DevOps automation overview
4. CI, continuous delivery, and continuous deployment
5. Jenkins architecture
6. Jenkins installation and configuration
7. GitHub integration
8. Pipeline architecture and stages
9. Automated testing
10. Infrastructure as Code and Terraform architecture
11. Terraform resource and configuration
12. Terraform execution procedure
13. Deployment and verification
14. Execution results and limitations
15. Browser-rendered health endpoint view
16. Results and observations
17. Conclusion
Execution evidence and scope
18. Complete implementation source
References

1. Introduction

Continuous Integration and Continuous Delivery/Deployment (CI/CD) automate the path from source change to a tested and deployable application. Jenkins provides an extensible automation server and Pipeline-as-Code model. Terraform describes and provisions infrastructure through declarative configuration. This assignment defines a Jenkins pipeline for the same Student Management Portal and uses Terraform's local provider to create a verifiable local artifact without incurring cloud costs.

2. Objectives

- Explain CI/CD concepts and the Jenkins controller/agent/pipeline model.
- Define a declarative Jenkinsfile with checkout, build, test, Terraform, deployment, and verification stages.
- Describe GitHub source integration through a Jenkins Pipeline job configured from source control.
- Use Terraform providers and resources to provision a real local file as an infrastructure-as-code demonstration.
- Distinguish pipeline source artifacts from actual Jenkins and Terraform execution evidence.

3. DevOps automation overview

Automation reduces manual repetition and creates auditable, repeatable delivery steps. A pipeline should fail fast when tests or validation fail, keep credentials out of source files and logs, and expose a clear record of each stage. The example is designed for a Linux Jenkins agent with Git, Node.js 22, Docker Engine, Terraform 1.5+, and curl installed.

4. CI, continuous delivery, and continuous deployment

Continuous Integration (CI) frequently integrates changes into a shared repository and runs automated build and test checks. **Continuous Delivery** keeps the application in a deployable state, with production release typically requiring an explicit approval. **Continuous Deployment** automatically releases every change that passes the defined pipeline to production. The Jenkinsfile here automates a local demonstration deployment; it does not imply production continuous deployment.

5. Jenkins architecture

Figure 3.1 — Jenkins pipeline architecture



The Jenkins controller stores job configuration and coordinates pipeline execution. An agent provides the workspace and tools required by the pipeline. The job is configured as a Pipeline from SCM so `checkout scm` retrieves the configured repository revision. The Jenkinsfile defines the stage sequence as code.

6. Jenkins installation and configuration

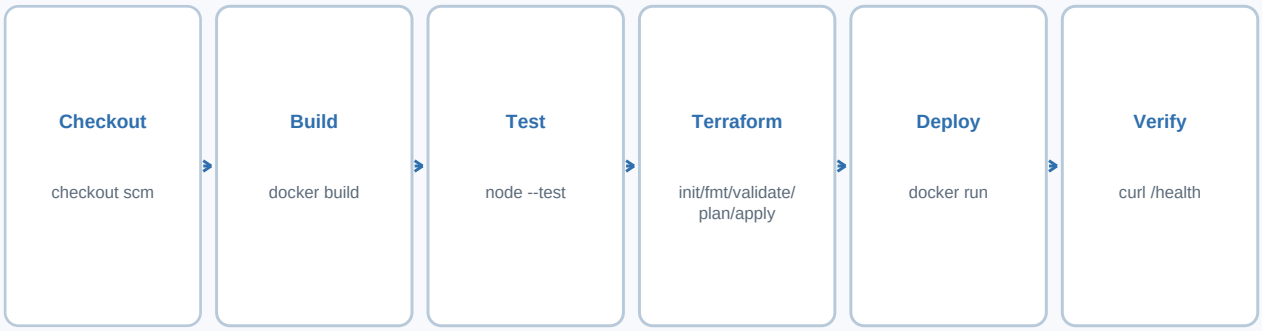
On a supported Linux host, install Jenkins using the official distribution-specific instructions, start the service, unlock the initial administrator account, and install the recommended plugins including Pipeline and Git. Configure a Linux agent with Git, Node.js, Docker, Terraform, and curl. Do not place passwords or tokens in the Jenkinsfile; use Jenkins Credentials and bind them only to stages that require them.

7. GitHub integration

Create a GitHub repository containing the project directories and Jenkinsfile. In Jenkins, create a Pipeline job and select “Pipeline script from SCM”, choose Git, enter the repository URL, select the branch, and set the script path to `IA-3-Jenkins-Terraform/Jenkinsfile` if the file is kept in that subdirectory. Configure a GitHub webhook or use SCM polling according to the environment. The repository URL was not provided for this execution, so this report does not claim a live GitHub connection or webhook delivery.

8. Pipeline architecture and stages

Figure 3.2 — Declarative pipeline stages



The Checkout stage retrieves the configured SCM revision. Build creates the container image. Test runs the Node.js built-in test runner. Terraform Init downloads and initializes the required provider; Format/Validate checks the HCL; Plan previews the local file resource; Apply creates or updates it. Deploy replaces a previous demo container and starts the new image. Verify retries the health endpoint. A post-always block removes the temporary container.

9. Automated testing

The application test suite uses Node.js' built-in `node:test` module and an isolated temporary data file. One test starts a server process, checks the health endpoint, reads students, registers a new student, and verifies the list grows. A second test confirms that an incomplete registration payload receives HTTP 400. These tests were executed locally and both passed.

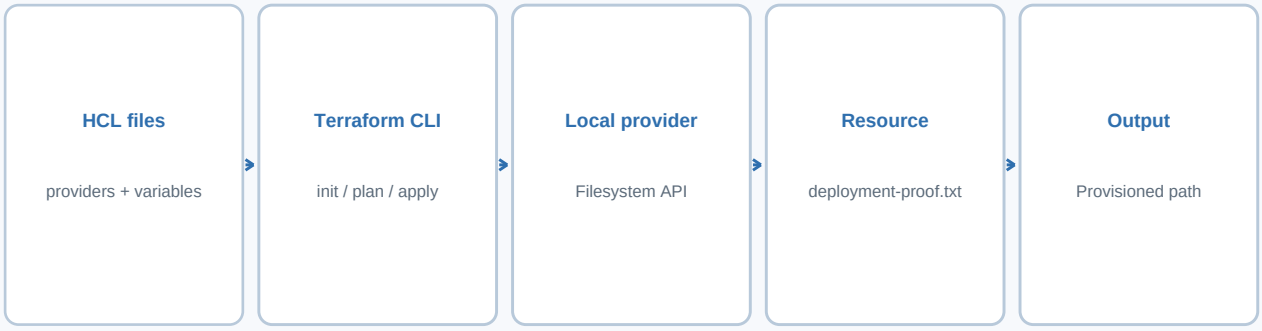
```
cd IA-2-Kubernetes/app
npm test

TAP version 13
# Subtest: health endpoint, student listing and registration
ok 1 - health endpoint, student listing and registration
# Subtest: registration rejects incomplete payload
ok 2 - registration rejects incomplete payload
1..2
# tests 2
# pass 2
# fail 0
```

10. Infrastructure as Code and Terraform architecture

Terraform uses a provider plugin to communicate with an infrastructure or platform API. Resources describe objects that Terraform creates, updates, and destroys. Variables parameterize configuration; outputs expose useful values. State records the mapping between configuration and managed objects. In this project, the HashiCorp local provider manages a local file, making the resource tangible without provisioning a cloud VM or incurring cloud usage charges.

Figure 3.3 — Terraform local provisioning



11. Terraform resource and configuration

The required provider block pins the local provider to the 2.5 minor series and requires Terraform 1.5 or newer. The `local_file.deployment_record` resource writes a deployment record into the Terraform module directory. The application name is a variable; the output returns the path of the managed file. The resource is a local filesystem artifact, not a cloud server, network, or Kubernetes resource.

12. Terraform execution procedure

```
cd IA-3-Jenkins-Terraform/terraform
terraform init -input=false
terraform fmt -check
terraform validate
terraform plan -out=tfplan
terraform apply -auto-approve tfplan
terraform output
cat deployment-proof.txt
terraform destroy -auto-approve
```

The plan is not proof of provisioning. A successful apply must create the file and the output must resolve to its path. Destroy removes the resource from the local filesystem. The pipeline leaves the resource in the Jenkins workspace as a build artifact; an ephemeral workspace may be cleaned by the Jenkins agent after the job.

13. Deployment and verification

After build, tests, and Terraform apply succeed, the Deploy stage removes any previous container with the demo name and starts the image with host port 3001 mapped to container port 3000. Verify requests `http://127.0.0.1:3001/health` with retries. The post action removes the container regardless of success or failure. This local deployment is a demonstration target, not a production rollout strategy.

14. Execution results and limitations

Verified: Node.js v22.16.0 and npm 10.9.2 were available; the two application tests passed; the portal health/list/register API and Nginx reverse proxy were tested independently; Git 2.47.3 and Java 21 were present.

Not verified: Jenkins, Docker Engine/CLI, and Terraform were unavailable. External downloads were blocked. No Jenkins dashboard/job, SCM checkout, pipeline stage visualization, Docker image, Terraform init/plan/apply, provisioned file, or container deployment was executed. No GitHub URL or credentials were supplied, and no remote integration is claimed.

15. Browser-rendered health endpoint view

The following image is a Chromium-rendered display of the health response collected from the running Node.js service. It is a formatted response view, not a screenshot of a Jenkins deployment or a live browser navigation to the service.

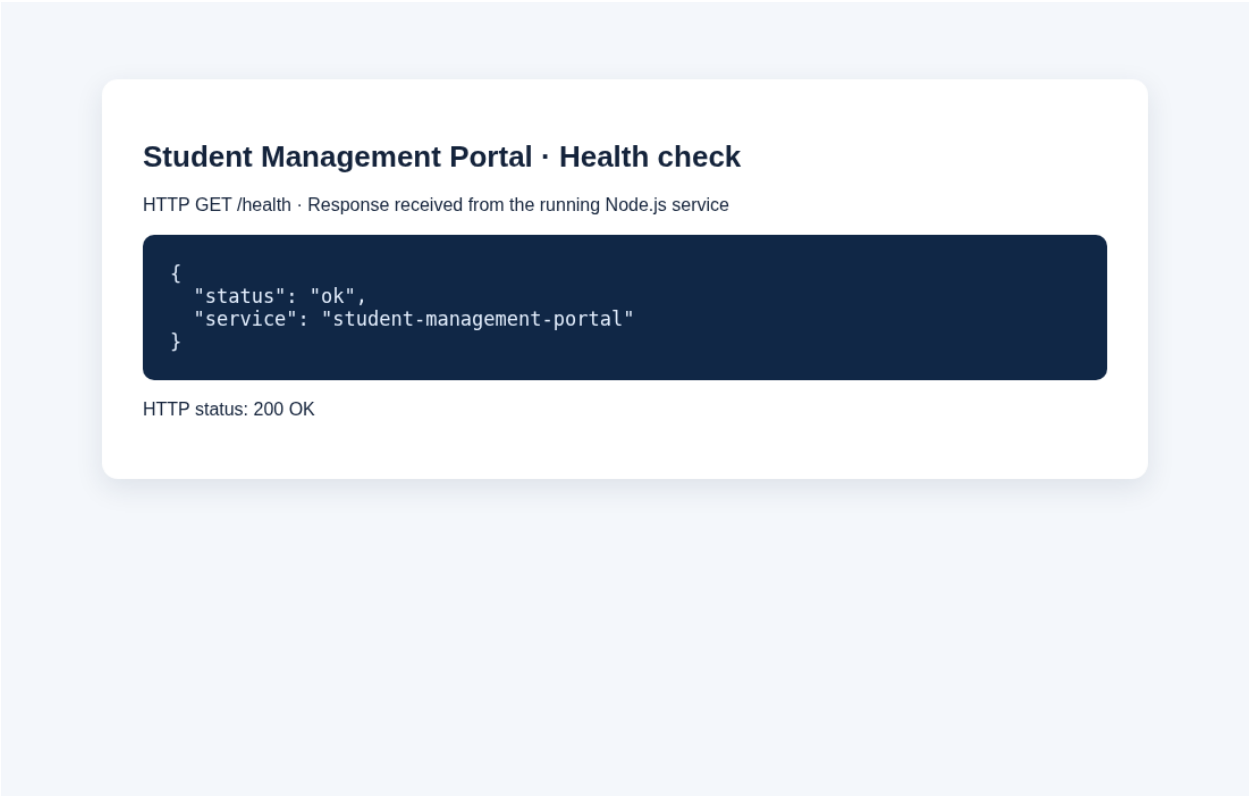


Figure 3.4 — HTTP health response view; Jenkins and Terraform stages were not executed.

16. Results and observations

- The declarative Jenkinsfile separates checkout, build, test, Terraform, deployment, and verification.
- The Terraform local provider provides a real, cost-free local provisioning target when Terraform and its provider can be installed.
- The test suite is independent of Jenkins and passes in the current environment.
- CI, continuous delivery, and continuous deployment are distinct operating models; the demo does not claim automatic production release.
- Jenkins SCM integration and Terraform provisioning remain pending execution on a configured Jenkins agent with required tools and repository access.

17. Conclusion

This assignment defines a complete CI/CD pipeline and Terraform local-resource configuration for the Student Management Portal. The Node.js application tests passed, but the infrastructure and pipeline stages could not be executed because the required tools and network access were unavailable. The report therefore provides the complete source and repeatable commands without claiming a successful Jenkins or Terraform run.

Execution evidence and scope

This report separates verified execution from implementation artifacts that could not be run in the available sandbox. The execution host was Debian GNU/Linux 13 with Node.js v22.16.0, npm 10.9.2, Nginx 1.26.3, Git 2.47.3 and OpenJDK 21. It was not the student's Windows 11/WSL2 laptop. External package downloads were blocked by DNS/network restrictions.

Evidence integrity: no fabricated terminal output, Ansible recap, Docker build, Kubernetes pod status, Jenkins stage result, Terraform apply, or GitHub connection is presented. The three images in these reports are browser-rendered interface/health views generated in Chromium from the portal markup; they are not screenshots of a live interactive desktop browser session. API and reverse-proxy checks are separately recorded from real HTTP requests.

- **Executed:** Node.js application tests (2 passed), direct HTTP checks, Nginx reverse-proxy check, Git and Java version checks.

- **Not executed:** Jenkins installation/dashboard/job, GitHub SCM checkout/webhook, Docker build/run, Terraform init/fmt/validate/plan/apply, local resource creation, pipeline visualization, or CI deployment verification.
- **Cost choice:** Terraform local provider is selected to avoid cloud infrastructure charges.

No PRN, roll number, faculty name, signature, or marks have been invented. The semester is left as “As applicable to the course” because no semester value was supplied.

18. Complete implementation source

18.1 Jenkinsfile

```
pipeline {
  agent any
  options {
    timestamps()
    disableConcurrentBuilds()
  }
  environment {
    APP_IMAGE = 'student-portal:ci'
    CONTAINER_NAME = 'student-portal-ci'
  }
  stages {
    stage('Checkout') {
      steps { checkout scm }
    }
    stage('Build') {
      steps {
        dir('IA-2-Kubernetes/app') {
          sh 'node --version'
          sh 'docker build -t ${APP_IMAGE} .'
        }
      }
    }
    stage('Test') {
      steps {
        dir('IA-2-Kubernetes/app') { sh 'npm test' }
      }
    }
    stage('Terraform Init') {
      steps { dir('IA-3-Jenkins-Terraform/terraform') { sh 'terraform init -input=false' } }
    }
    stage('Terraform Format and Validate') {
      steps {
        dir('IA-3-Jenkins-Terraform/terraform') {
          sh 'terraform fmt -check'
          sh 'terraform validate'
        }
      }
    }
    stage('Terraform Plan') {
      steps { dir('IA-3-Jenkins-Terraform/terraform') { sh 'terraform plan -out=tfplan' } }
    }
    stage('Terraform Apply') {
      steps {
        dir('IA-3-Jenkins-Terraform/terraform') {
          sh 'terraform apply -auto-approve tfplan'
        }
      }
    }
    stage('Deploy') {
      steps {
        sh 'docker rm -f ${CONTAINER_NAME} || true'
        sh 'docker run -d --name ${CONTAINER_NAME} -p 3001:3000 ${APP_IMAGE}'
      }
    }
    stage('Verify') {
      steps { sh 'curl --fail --retry 10 --retry-delay 2 http://127.0.0.1:3001/health' }
    }
  }
  post {
    always {
      sh 'docker rm -f ${CONTAINER_NAME} || true'
    }
  }
}
```

18.2 Terraform providers.tf

```
terraform {
  required_version = ">= 1.5.0"
  required_providers {
    local = {
      source  = "hashicorp/local"
      version = "~> 2.5"
    }
  }
}
```

18.3 Terraform variables.tf

```
variable "application_name" {
  description = "Application identifier recorded in the provisioned artifact."
  type        = string
  default     = "student-management-portal"
}
```

18.4 Terraform main.tf

```
resource "local_file" "deployment_record" {
  filename = "${path.module}/deployment-proof.txt"
  content  = <<-EOT
    application=${var.application_name}
    provisioned_by=Terraform local provider
    purpose=Local infrastructure provisioning demonstration
  EOT
  file_permission = "0644"
}
```

18.5 Terraform outputs.tf

```
output "provisioned_artifact_path" {
  description = "Absolute path of the file provisioned by Terraform."
  value       = local_file.deployment_record.filename
}
```

18.6 package.json and test script

```
{
  "name": "student-management-portal",
  "version": "1.0.0",
  "private": true,
  "scripts": {
    "start": "node server.js",
    "test": "node --test"
  }
}
```

18.7 API test suite

```
const test = require('node:test');
const assert = require('node:assert/strict');
const { spawn } = require('node:child_process');
const fs = require('node:fs');
const os = require('node:os');
const path = require('node:path');
const http = require('node:http');

function request(port, method, route, body) {
  return new Promise((resolve, reject) => {
    const data = body ? Buffer.from(JSON.stringify(body)) : undefined;
    const req =
http.request({hostname:'127.0.0.1',port,path:route,method,headers:data?{'Content-Type':'application/json','Conte
nt-Length':data.length}:{}}, res => {
      let raw=''; res.on('data',c=>raw+=c);
res.on('end',()=>resolve({status:res.statusCode,body:raw?JSON.parse(raw):null}));
    });
    req.on('error',reject); if(data) req.write(data); req.end();
  });
}
test('health endpoint, student listing and registration', async t => {
  const port=31000+Math.floor(Math.random()*1000);
  const dataPath=path.join(os.tmpdir(),`students-${process.pid}-${Date.now()}.json`);
  const
child=spawn(process.execPath,[path.join(__dirname,'..','server.js')],{env:{...process.env,PORT:String(port),DATA
_PATH:dataPath},stdio:'ignore'});
  t.after(()=>{child.kill(); try{fs.unlinkSync(dataPath)}catch{}});
  let ready=false;
  for(let i=0;i<30;i++){try{await request(port,'GET','/health');ready=true;break}catch{await new
Promise(r=>setTimeout(r,100))}}
  assert.equal(ready,true,'server should start');
  const health=await request(port,'GET','/health');
  assert.equal(health.status,200); assert.equal(health.body.status,'ok');
  const before=await request(port,'GET','/api/students');
  assert.equal(before.status,200); assert.equal(Array.isArray(before.body),true);
  const created=await request(port,'POST','/api/students',{name:'Test
Student',email:'test@example.edu',course:'Computer Engineering',year:2});
  assert.equal(created.status,201); assert.equal(created.body.name,'Test Student');
  const after=await request(port,'GET','/api/students');
  assert.equal(after.body.length,before.body.length+1);
});
test('registration rejects incomplete payload', async t => {
  const port=32000+Math.floor(Math.random()*1000);
  const dataPath=path.join(os.tmpdir(),`students-${process.pid}-${Date.now()}.json`);
  const
child=spawn(process.execPath,[path.join(__dirname,'..','server.js')],{env:{...process.env,PORT:String(port),DATA
_PATH:dataPath},stdio:'ignore'});
  t.after(()=>{child.kill(); try{fs.unlinkSync(dataPath)}catch{}});
  let ready=false;
  for(let i=0;i<30;i++){try{await request(port,'GET','/health');ready=true;break}catch{await new
Promise(r=>setTimeout(r,100))}}
  assert.equal(ready,true);
  const response=await request(port,'POST','/api/students',{name:'Missing fields'});
  assert.equal(response.status,400);
});
```

References

The following primary documentation sources are provided for the technologies and interfaces used in this report.

• **Jenkins Pipeline documentation** — https://www.jenkins.io/doc/book/pipeline/

• **Jenkins Git plugin** — https://plugins.jenkins.io/git/

• **GitHub webhooks documentation** — https://docs.github.com/en/webhooks

• **Terraform language documentation** — https://developer.hashicorp.com/terraform/language

• **Terraform local provider** — https://registry.terraform.io/providers/hashicorp/local/latest/docs

• **Terraform CLI commands** — https://developer.hashicorp.com/terraform/cli/commands

• **Node.js test runner** — https://nodejs.org/docs/latest/api/test.html

• **Docker build documentation** — https://docs.docker.com/build/

Step-by-step visual procedure references

The following visuals are structured walkthrough references for the setup, configuration, execution, and verification steps. They are provided to make the sequence and expected artifacts easier to follow.

Visuals are marked as reference views. Use the actual local command output and application screens when recording your own execution evidence.

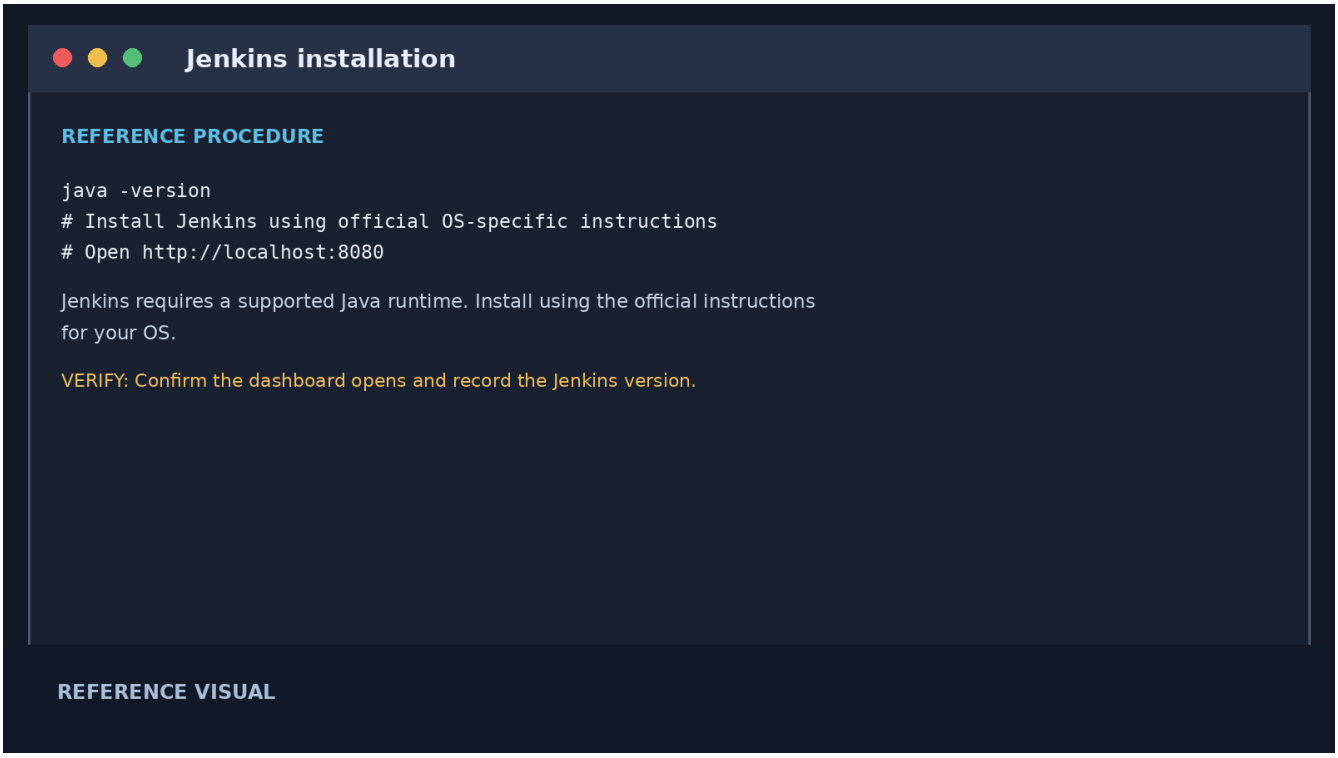


Figure IA3-A — Jenkins installation and dashboard access procedure.



Figure IA3-B — Example declarative Jenkins pipeline stages.

Terraform local resource

```
terraform {
  required_providers {
    local = { source = "hashicorp/local" }
  }
}

resource "local_file" "deployment_marker" {
  filename = "${path.module}/build/deployed.txt"
  content  = "Student Portal deployment marker"
}
```

REFERENCE VISUAL

Figure IA3-C — Example Terraform local resource configuration.

Pipeline and resource verification

REFERENCE PROCEDURE

```
terraform state list
cat build/deployed.txt
# In Jenkins, inspect completed build and stage results
```

A completed Jenkins run and created file are required to prove provisioning; a plan alone is insufficient.

VERIFY: Verify all stages, Terraform state, and the provisioned local file.

REFERENCE VISUAL

Figure IA3-D — Pipeline and provisioned-resource verification steps.

GitHub repository review and external visual references

A public-profile review was performed for the DevOps assignment artifacts. The repository links below are the relevant matches found; the absence of a match means the requested assignment source is not currently visible in the reviewed public repositories.

jenkinsDemo — jenkinsDemo

Contains a Jenkinsfile and jenkin.txt. The Jenkinsfile is a minimal Hello-stage pipeline only; it does not show checkout, build, test, Terraform or deployment stages.

terraform-provider-nomad — terraform-provider-nomad

Public fork of hashicorp/terraform-provider-nomad; not the Student Management Portal Terraform project.

terraform-plugin-testing — terraform-plugin-testing

Public fork of hashicorp/terraform-plugin-testing; not the Student Management Portal Terraform project.

Screenshot and documentation references checked

Medium articles were checked for step-by-step visual examples, including Ansible ping/playbook evidence and Jenkins/Docker/Terraform/Kubernetes pipeline views. Official documentation was also checked for accurate command sequences and UI workflows. These sources are learning references, not evidence of execution in this project.

- Ansible official inventory guide
- Docker official container tutorial
- Kubernetes official Pods tutorial
- Jenkins official Pipeline setup guide
- Medium — Ansible deployment with successful ping and playbook examples
- Medium — Jenkins, Docker, Terraform and Kubernetes deployment walkthrough
- Medium — Step-by-step Kubernetes, Terraform and Jenkins guide

Google/search review: web search results were used to locate the Medium articles and official product documentation above. Google is a search index, not the original source of the screenshots; the cited page is the source to consult.